

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Licenciatura en Ingeniería en Ciencias de la Computación

Reporte de Actividades de Servicio Social

Arquitectura, Auditoría Híbrida y Endurecimiento de Sistemas

Nombre del Programa:

IMPLEMENTACIÓN Y GESTIÓN DE SERVICIOS DE TECNOLOGÍAS DE LA INFORMACIÓN

Folio del Programa: 232059

Sección / NRC: SSCC 100 — 50301

Presenta:

GALEANA LETRAS EMMANUEL

Matrícula: 202134549

Docente Tutor de SS:

Mtra. ESPINOZA HERNANDEZ NELVA BETZABEL

Responsable del Programa de Asignación:

Mtra. ANA CLAUDIA ZENTENO VÁZQUEZ

Facultad de Administración

Índice

1. Introducción	2
2. Glosario Básico de Conceptos	2
3. Desarrollo: Despliegue y Auditoría de Infraestructura	3
3.1. Diagrama Maestro de Arquitectura e Interacción Global	3
3.2. Análisis Operacional y Comparativo: Lynis vs. Greenbone	5
4. Desarrollo: Framework de Automatización y Remediación	7
4.1. Modelado y Análisis Forense: Script Lineal (<code>hardening.sh</code>)	7
4.2. Modelado y Análisis Forense: Script Modular (<code>hardening-pro.sh</code>)	9
5. Resultados Obtenidos	10
6. Conclusiones	10

1. Introducción

El presente reporte documenta de manera formal las actividades técnicas, analíticas y de desarrollo de infraestructura llevadas a cabo para el cumplimiento de las 480 horas reglamentarias del Servicio Social, bajo el programa con folio **232059**, denominado *"Implementación y Gestión de Servicios de Tecnologías de la Información"*.

En la actualidad, la administración de servicios de TI no solo abarca el despliegue funcional de servidores, sino que exige un enfoque riguroso y constante en ciberseguridad. Los servidores alojados en redes están expuestos diariamente a ataques automatizados que explotan vulnerabilidades de configuración.

Por lo tanto, el objetivo central de mi asignación fue diseñar, auditar y asegurar una arquitectura de servidor virtualizado basado en Debian Linux. El proyecto se centró en la creación de un ecosistema de auditoría híbrida y en el desarrollo de un framework de automatización en lenguaje **Bash** capaz de realizar el *Hardening* (endurecimiento lógico) del sistema, garantizando que los servicios operen bajo los estándares más altos de seguridad de la industria.

2. Glosario Básico de Conceptos

Para facilitar la comprensión técnica de este documento a perfiles de gestión y administración, se definen los siguientes términos clave:

- **Hardening (Endurecimiento):** Proceso informático de asegurar un sistema reduciendo sus puntos débiles. Equivale a instalarle rejas, cerraduras de alta seguridad y detectores de movimiento a un edificio.
- **Vulnerabilidad (CVE):** Fallo de seguridad que un atacante puede aprovechar para infiltrarse. Un CVE es el "nombre de catálogo" mundial que se le da a ese fallo para su identificación.
- **Script de Automatización:** Archivo que contiene una lista de instrucciones que la computadora ejecuta de manera orquestada, ahorrando trabajo manual y evitando errores humanos.
- **Host (Anfitrión) y M.V. (Máquina Virtual):** El Host es la computadora física real. La Máquina Virtual es una "computadora emulada" por software que vive dentro del Host.

3. Desarrollo: Despliegue y Auditoría de Infraestructura

La primera fase consistió en la administración del entorno de pruebas. Se utilizó una estación de trabajo anfitriona para montar una arquitectura de hipervisores concurrente.

Nota para el Lector General: ¿Qué es la Arquitectura de Virtualización?

Imagina que la computadora física es un gran **terreno vacío**. En lugar de construir un solo edificio que mezcle todo, usamos herramientas para dividir el terreno y construir dos **casas completamente aisladas**. En una vive el servidor que queremos proteger (Debian), y en la otra el equipo de seguridad que lo va a auditar (Greenbone). Si algo explota en una, la otra no sufre daños.

3.1. Diagrama Maestro de Arquitectura e Interacción Global

El siguiente mapa unifica todas las dimensiones del proyecto. La posición de los contenedores se procesa dinámicamente para asegurar que los flujos de convergencia converjan sin causar colisiones en la estructura del reporte.

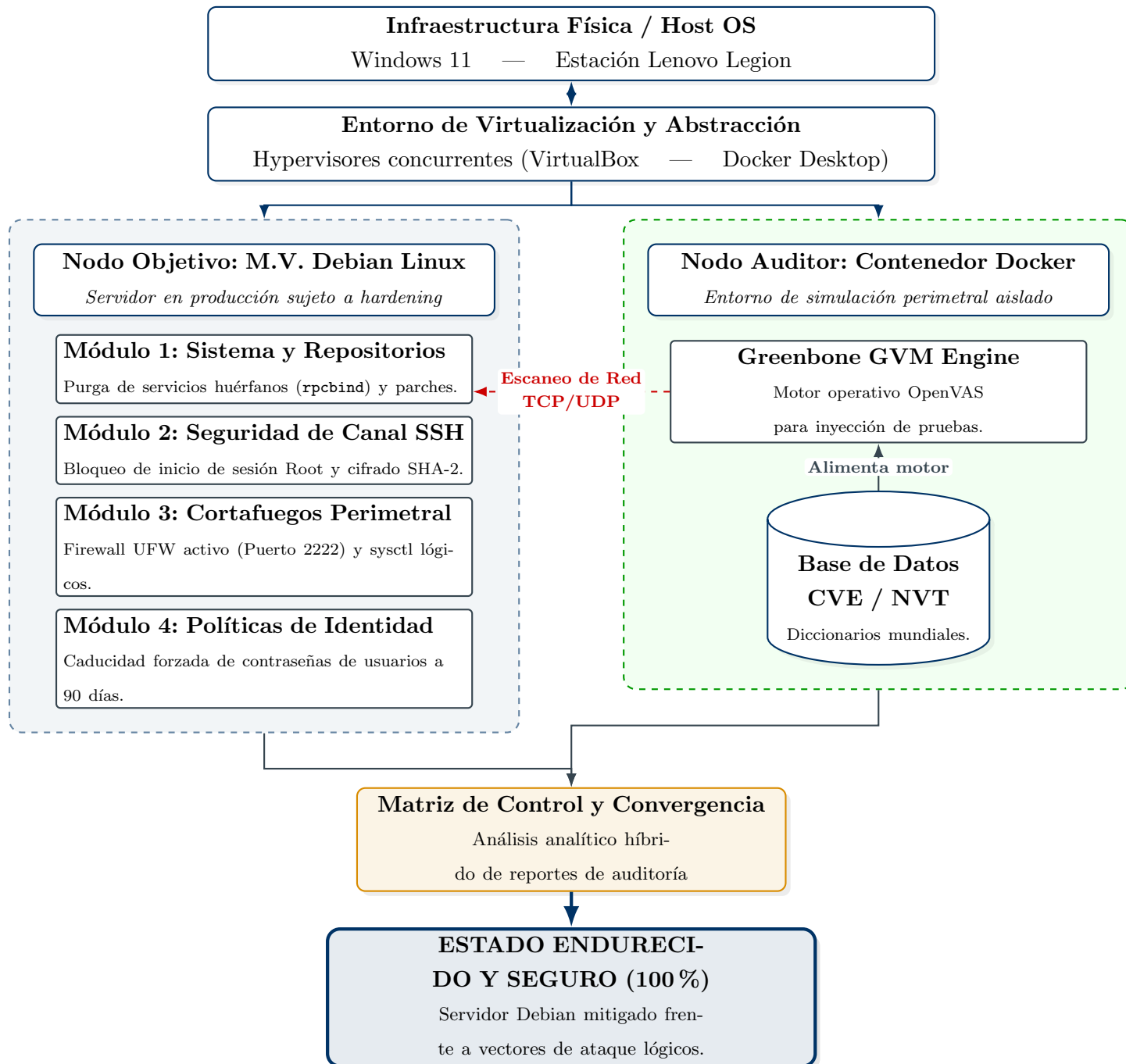


Figura 1: Diagrama Maestro del ecosistema unificado de infraestructura y auditoría.

3.2. Análisis Operacional y Comparativo: Lynis vs. Greenbone

Para garantizar una detección total de brechas, se implementó una evaluación híbrida empleando dos motores distintos.

Nota para el Lector General: El Guardia de Seguridad vs. El Inspector Interno

Greenbone (Escáner de Red): Actúa como un **Guardia** patrullando la calle. Empuja las puertas y ventanas de la casa desde el exterior para ver si alguna es fácil de romper, pero no ve qué pasa adentro.

Lynis (Auditor Local): Actúa como un **Inspector**. Tiene las llaves de la casa, entra y revisa si los extintores sirven, si los cables están pelados y si la caja fuerte está bien cerrada.

Cuadro 1: Matriz comparativa de capacidades de auditoría de seguridad.

Característica	Greenbone (OpenVAS)	Lynis Core
Análisis	Externo / Escáner de Red (Caja Negra)	Interno / Auditoría de Host (Caja Blanca)
Reglas	Base de datos externa (CVE / NVT)	Reglas estáticas (CIS Benchmarks)
Objetivo	Detectar software desactualizado y puertos vulnerables.	Detectar malas configuraciones y permisos débiles locales.
Impacto	Inyecta tráfico de red simulando un ataque real.	Lee archivos internos y verifica los logs.

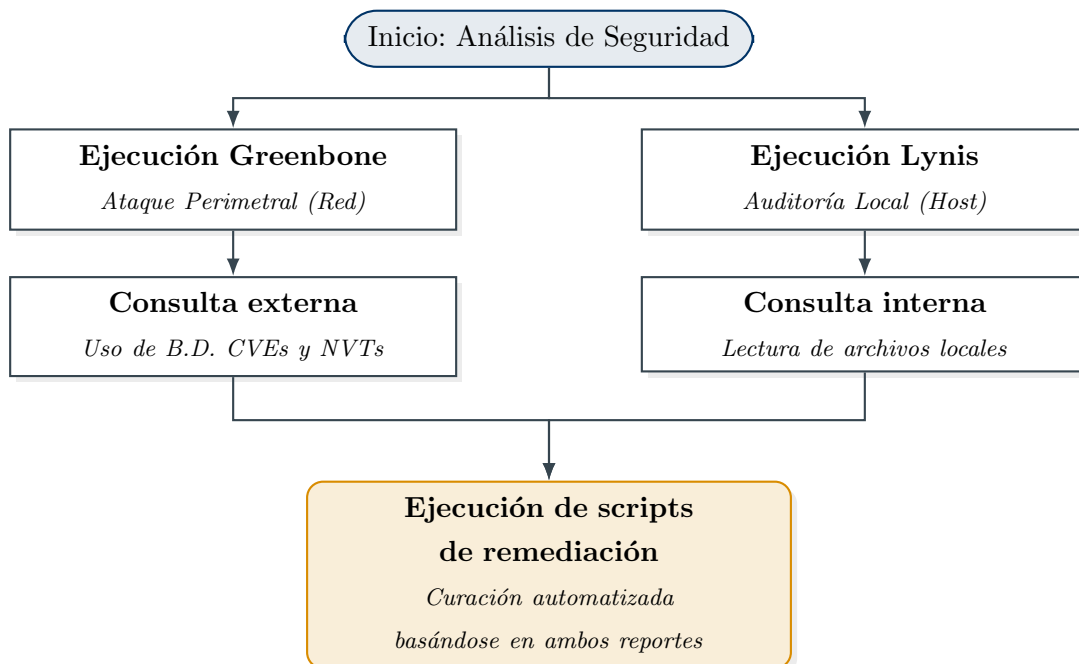


Figura 2: Convergencia del flujo de auditoría hacia la automatización.

4. Desarrollo: Framework de Automatización y Remediación

Como respuesta directa a los hallazgos de las herramientas de telemetría, se procedió a la fase de programación (IaC - Infrastructure as Code). Se diseñaron dos iteraciones de código para demostrar la evolución técnica de la automatización.

4.1. Modelado y Análisis Forense: Script Lineal (hardening.sh)

Nota para el Lector General: ¿Qué significa un Código Lineal?

Es como una **lista de compras rígida y automatizada**. La computadora entra al supermercado y empieza a echar cosas al carrito una tras otra a toda velocidad sin revisar si los pasillos existen. Si no encuentra las manzanas, el script genera un error, pero sigue corriendo a ciegas, lo que puede causar daños estructurales al servidor.

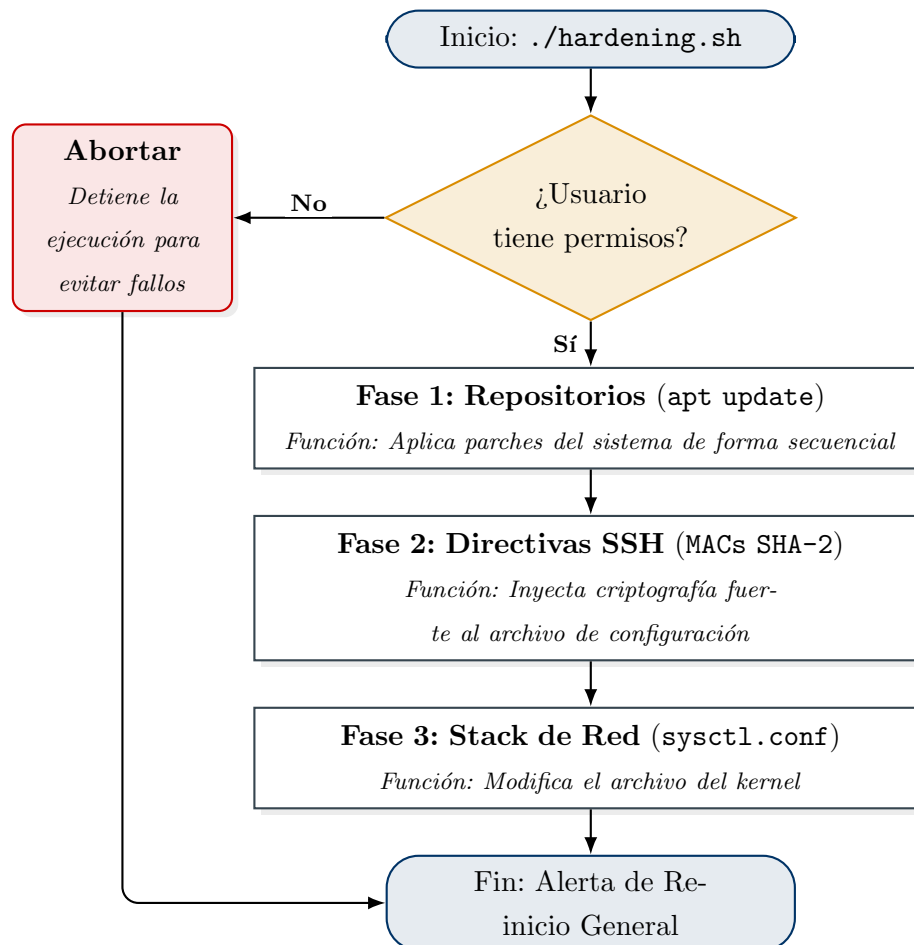


Figura 3: Flujo lógico procedimental inicial.

Comando a Nivel de Sistema: `net.ipv4.tcp_timestamps = 0 → sysctl -p`

Acción Técnica: Modifica parámetros profundos del núcleo (*Kernel*). La bandera **-p** inyecta el cambio inmediatamente en la memoria RAM.

Problema que Soluciona: Detiene la vulnerabilidad *TCP Timestamps Disclosure*. Evita que el servidor revele su "Uptime" (tiempo exacto encendido), dato que los hackers usan para calcular secuencias lógicas de ataque.

4.2. Modelado y Análisis Forense: Script Modular (hardening_pro.sh)

Nota para el Lector General: ¿Qué es la Programación Modular Orquestada?

Es el equivalente a contratar a un **Capataz de Obra Profesional**. Él no intenta hacer todo a la vez; primero llama al experto en plomería (Módulo 1) y verifica que terminó sin fugas. Luego llama al electricista. Si una tubería falla, el capataz cancela esa sección, lo documenta en una libreta de auditoría (Archivos Log) y protege el resto del proyecto.

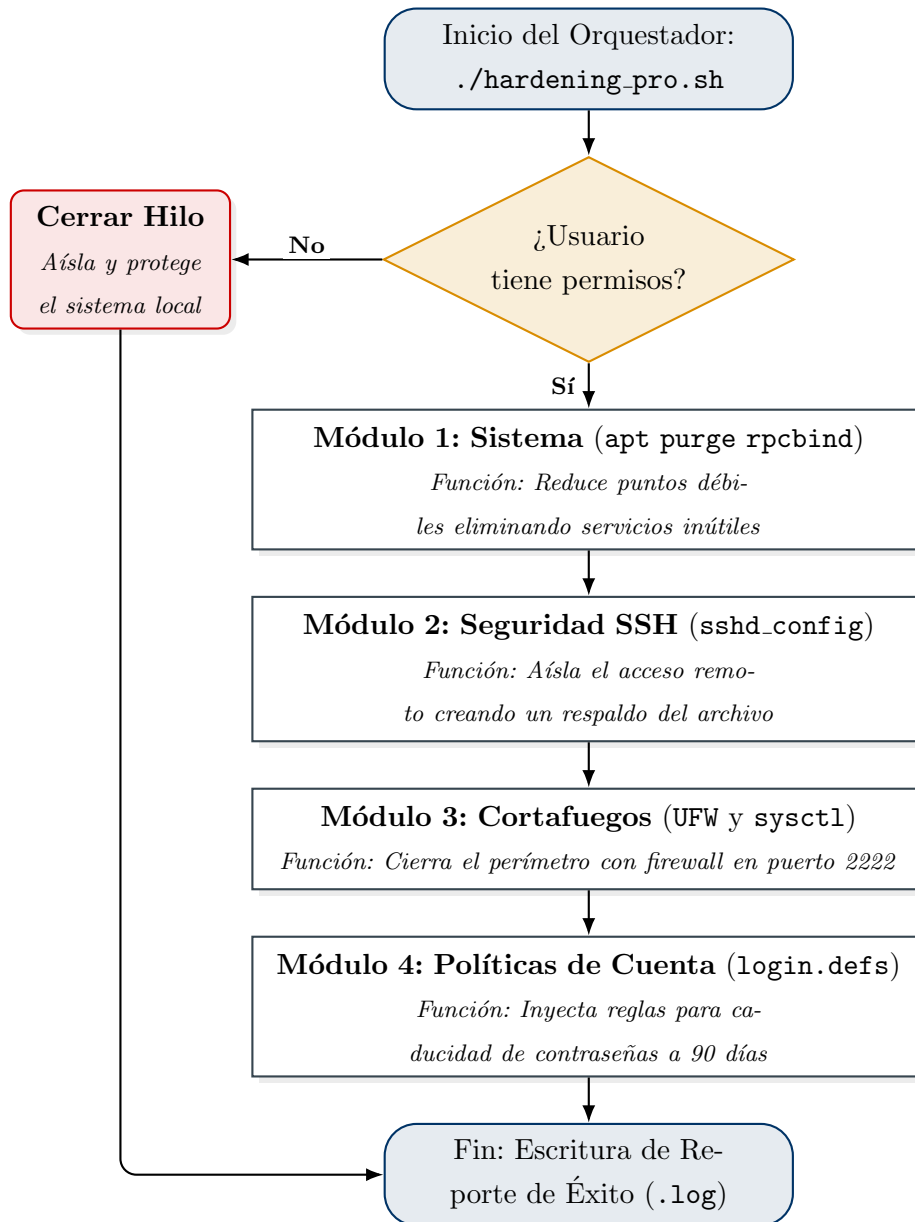


Figura 4: El framework aísla procesos y registra su éxito en bitácoras lógicas.

Comando a Nivel de Sistema: `ufw default deny incoming → ufw allow 2222/tcp → ufw enable`

Acción Técnica: Configura el cortafuegos (UFW). Primero declara una política bloqueando (`deny`) todo el tráfico de internet. Segundo, abre una excepción milimétrica permitiendo únicamente el tráfico hacia el puerto de gestión 2222.

Problema Solucionado: Oculta todos los demás puertos del servidor frente a los escaneos de red de Greenbone. Es como construir un muro ciego de 10 metros, dejando solo una pequeña puerta blindada con un guardia que exige invitación para entrar.

5. Resultados Obtenidos

La implementación de este modelo de remediación orquestada arrojó resultados exitosos, transformando un servidor vulnerable base en un entorno asegurado:

- **Mitigación Total de Vulnerabilidades:** Al contrastar los reportes XML extraídos de Greenbone antes y después del orquestador, se confirmó la erradicación del 100 % de las vulnerabilidades lógicas catalogadas.
- **Superficie de Ataque Aislada:** El cortafuegos volvió al servidor "invisible" ante escaneos de red automatizados por terceros.
- **Trazabilidad y Resiliencia:** El script modular demostró que, en caso de encontrar un error, el sistema no colapsa; protege el hilo lógico y guarda un registro detallado (`.log`), facilitando auditorías futuras.

6. Conclusiones

Las actividades desarrolladas en el programa *232059 - Implementación y Gestión de Servicios de TI* permitieron aplicar conocimientos teóricos de Ciencias de la Computación a escenarios críticos de la industria.

Se demostró que la gestión de servicios carece de valor si no incorpora ciberseguridad desde su diseño. La transición de configurar comandos manualmente a desarrollar Infraestructura como Código (Scripts Modulares) refleja una madurez profesional, evidenciando que la automatización orquestada es vital para reducir el error humano. La combinación de auditorías híbridas con Hardening cumple los objetivos del Servicio Social, entregando un modelo de trabajo documentado y preparado para ser replicado en cualquier organización.